

• DATA ANALYTICS PORTFOLIO

Pizza Sales Analysis in SQL

An end-to-end MySQL case study answering 13 real business questions across a pizza restaurant's order data.

MySQL / 13 Queries / 4 Tables



The Project

A pizza restaurant tracks every order, item, and price. This analysis turns that raw transactional data into clear business insight using only SQL.

13

Business Questions

From simple counts to ranked revenue breakdowns.

4

Relational Tables

Orders, order details, pizzas, and pizza types.

3

Skill Tiers

Basic, intermediate, and advanced SQL techniques.

Database Schema

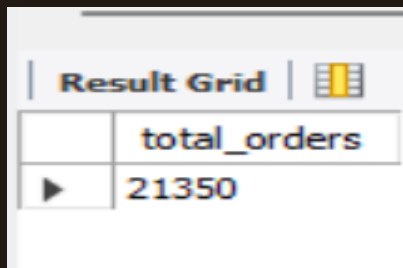
orders	order_details	pizzas	pizza_types
<code>order_id</code> PK <code>order_date</code> <code>order_time</code>	<code>order_details_id</code> PK <code>order_id</code> FK <code>pizza_id</code> FK quantity	<code>pizza_id</code> PK <code>pizza_type_id</code> FK size price	<code>pizza_type_id</code> PK name category ingredients

The four tables connect through foreign keys, letting a single query trace any sale from `order` to `pizza type`, price, and category.

How many orders were placed in total?

q1.sql

```
SELECT COUNT(order_id) AS total_orders  
FROM orders;
```



The screenshot shows a 'Result Grid' window with a single row of data. The column header is 'total_orders' and the value is '21350'. There is a play button icon in the first column of the data row.

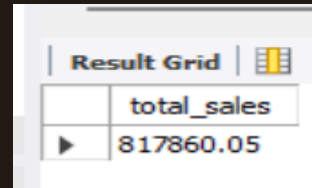
	total_orders
▶	21350

A single aggregate over the orders table establishes the baseline volume the rest of the analysis builds on.

What is the total revenue from pizza sales?

q2.sql

```
SELECT ROUND(SUM(order_details.quantity * pizzas.price),  
2) AS total_sales  
FROM order_details  
JOIN pizzas  
  ON pizzas.pizza_id = order_details.pizza_id;
```



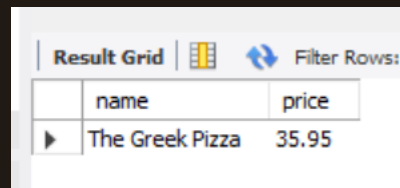
Result Grid	
	total_sales
▶	817860.05

Joining line items to prices and multiplying quantity by price gives true revenue, rounded to two decimals for currency.

Which pizza carries the highest price?

q3.sql

```
SELECT pizza_types.name, pizzas.price
FROM pizza_types
JOIN pizzas
    ON pizza_types.pizza_type_id = pizzas.pizza_type_id
ORDER BY pizzas.price DESC
LIMIT 1;
```



The screenshot shows a 'Result Grid' window with a 'Filter Rows' button. It displays a single row of data from a query. The columns are 'name' and 'price'. The row contains the text 'The Greek Pizza' and the value '35.95'.

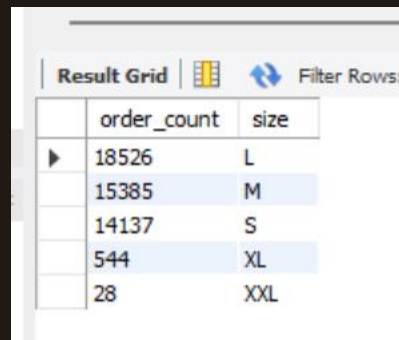
	name	price
▶	The Greek Pizza	35.95

Sorting prices in descending order and capping at one row isolates the single most expensive pizza on the menu.

Which pizza size is ordered most often?

q4.sql

```
SELECT COUNT(order_details.order_details_id) AS order_count,  
       pizzas.size  
FROM order_details  
JOIN pizzas  
    ON pizzas.pizza_id = order_details.pizza_id  
GROUP BY pizzas.size  
ORDER BY order_count DESC;
```



The screenshot shows a 'Result Grid' window with a table containing the results of the SQL query. The table has two columns: 'order_count' and 'size'. The rows are ordered by 'order_count' in descending order. The data is as follows:

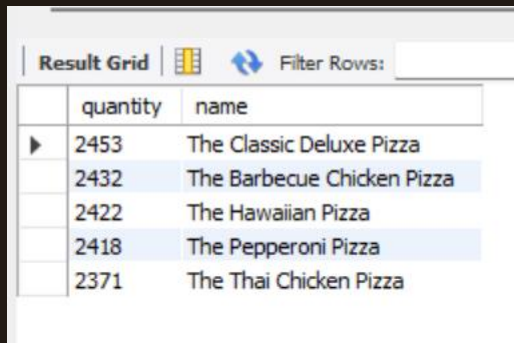
order_count	size
18526	L
15385	M
14137	S
544	XL
28	XXL

Grouping by size and counting line items reveals the customer's preferred portion, guiding inventory and pricing.

What are the top 5 most ordered pizza types?

q5.sql

```
SELECT SUM(order_details.quantity) AS quantity,  
       pizza_types.name  
  
FROM pizza_types  
JOIN pizzas  
    ON pizza_types.pizza_type_id = pizzas.pizza_type_id  
JOIN order_details  
    ON order_details.pizza_id = pizzas.pizza_id  
GROUP BY pizza_types.name  
ORDER BY quantity DESC  
LIMIT 5;
```



The screenshot shows a database interface with a 'Result Grid' tab selected. It displays the results of the SQL query, showing the top 5 most ordered pizza types by total quantity sold. The columns are 'quantity' and 'name'. The rows are sorted in descending order of quantity.

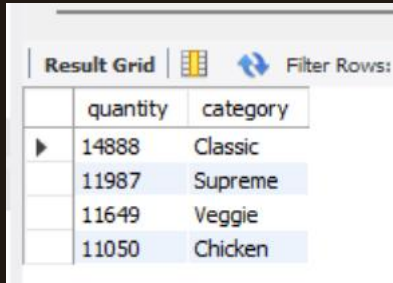
	quantity	name
▶	2453	The Classic Deluxe Pizza
	2432	The Barbecue Chicken Pizza
	2422	The Hawaiian Pizza
	2418	The Pepperoni Pizza
	2371	The Thai Chicken Pizza

A three-table join links sales back to pizza names, ranking the best-selling menu items by total quantity sold.

How much does each pizza category sell?

q6.sql

```
SELECT SUM(order_details.quantity) AS quantity,  
       pizza_types.category  
FROM pizza_types  
JOIN pizzas  
  ON pizza_types.pizza_type_id = pizzas.pizza_type_id  
JOIN order_details  
  ON order_details.pizza_id = pizzas.pizza_id  
GROUP BY pizza_types.category  
ORDER BY quantity DESC;
```



The screenshot shows a 'Result Grid' window with a table containing the results of the SQL query. The table has two columns: 'quantity' and 'category'. The rows are ordered by quantity in descending order. The first row is 'Classic' with a quantity of 14888, followed by 'Supreme' (11987), 'Veggie' (11649), and 'Chicken' (11050). There is a 'Filter Rows' button in the top right corner of the grid.

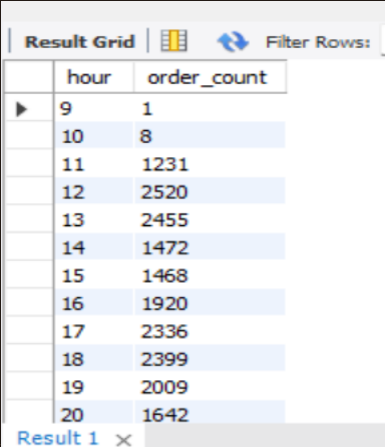
	quantity	category
▶	14888	Classic
	11987	Supreme
	11649	Veggie
	11050	Chicken

Rolling sales up to the category level shows which menu sections drive volume, and which underperform.

How are orders distributed across the day?

q7.sql

```
SELECT COUNT(order_id) AS order_count,  
       HOUR(order_time) AS hour  
FROM orders  
GROUP BY HOUR(order_time)  
ORDER BY hour;
```



The screenshot shows a database query result grid with two columns: 'hour' and 'order_count'. The data is sorted by hour from 9 to 20. The peak order count is 2520 at hour 12.

hour	order_count
9	1
10	8
11	1231
12	2520
13	2455
14	1472
15	1468
16	1920
17	2336
18	2399
19	2009
20	1642

Extracting the hour from each timestamp exposes peak service windows, informing staffing and prep schedules.

Join relevant tables to find the category wise distribution of pizza

q7.sql

```
SELECT COUNT (pizza_types.category) AS countpizzas, ,  
pizza_types.category  
FROM pizza_types  
GROUP BY pizza_types.category;
```



The screenshot shows a 'Result Grid' window with a table containing the results of the SQL query. The table has two columns: 'countpizzas' and 'category'. The data is as follows:

	countpizzas	category
▶	6	Chicken
	8	Classic
	9	Supreme
	9	Veggie

Extracting the count of each pizza category by grouping the 'pizza_types.category' and applying count o them.

What is the average number of pizzas per day?

q9.sql

```
SELECT ROUND(AVG(quantity), 0) AS avg_pizzas_per_day
FROM
( SELECT orders.order_date,
        SUM(order_details.quantity) AS quantity
  FROM orders
 JOIN order_details
   ON orders.order_id = order_details.order_id
 GROUP BY orders.order_date
) AS daily_totals;
```

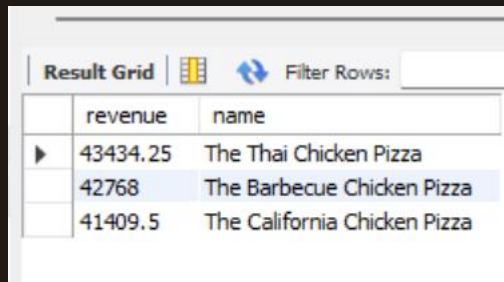
Result Grid		Filter Rows:
	avg_pizza_ordered_per_day	
▶	138	

A derived table first sums pizzas per day, then the outer query averages those daily totals — a two-step aggregation.

Which 3 pizza types generate the most revenue?

q10.sql

```
SELECT pizza_types.name,  
       SUM(order_details.quantity * pizzas.price) AS revenue  
FROM pizza_types  
JOIN pizzas  
  ON pizza_types.pizza_type_id = pizzas.pizza_type_id  
JOIN order_details  
  ON order_details.pizza_id = pizzas.pizza_id  
GROUP BY pizza_types.name  
ORDER BY revenue DESC  
LIMIT 3;
```



The screenshot shows a database interface with a 'Result Grid' tab selected. It displays a table with two columns: 'revenue' and 'name'. The table contains three rows of data, sorted in descending order of revenue. The first row is 'The Thai Chicken Pizza' with a revenue of 43434.25. The second row is 'The Barbecue Chicken Pizza' with a revenue of 42768. The third row is 'The California Chicken Pizza' with a revenue of 41409.5. There are navigation icons and a 'Filter Rows' input field at the top of the grid.

	revenue	name
▶	43434.25	The Thai Chicken Pizza
	42768	The Barbecue Chicken Pizza
	41409.5	The California Chicken Pizza

Ranking by revenue rather than volume surfaces the items that matter most to the bottom line.

What share of revenue comes from each category?

```
q11.sql

SELECT pizza_types.category,
       ROUND(SUM(order_details.quantity * pizzas.price)
 / (SELECT SUM(order_details.quantity * pizzas.price)
     FROM order_details
     JOIN
pizzas ON pizzas.pizza_id = order_details.pizza_id) * 100,
2) AS revenue_pct
FROM pizza_types
JOIN pizzas
  ON pizza_types.pizza_type_id = pizzas.pizza_type_id
JOIN order_details
  ON order_details.pizza_id = pizzas.pizza_id
GROUP BY pizza_types.category
ORDER BY revenue_pct DESC;
```



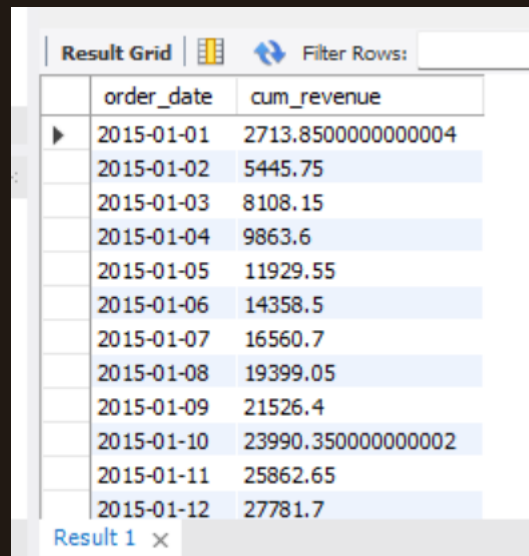
The screenshot shows a 'Result Grid' window with a table of query results. The table has two columns: 'category' and 'revenue'. There are four rows of data. The first row is 'Classic' with a revenue of 26.91. The second row is 'Supreme' with a revenue of 25.46. The third row is 'Chicken' with a revenue of 23.96. The fourth row is 'Veggie' with a revenue of 23.68. The window also includes a 'Filter Rows' button and a small icon of a grid.

	category	revenue
▶	Classic	26.91
	Supreme	25.46
	Chicken	23.96
	Veggie	23.68

How does revenue accumulate over time?

q12.sql

```
SELECT order_date,  
       SUM(revenue) OVER (ORDER BY order_date)  
       AS cumulative_revenue  
FROM  
( SELECT orders.order_date,  
        SUM(order_details.quantity * pizzas.price) AS revenue  
  FROM order_details  
 JOIN pizzas  
   ON order_details.pizza_id = pizzas.pizza_id  
 JOIN orders  
   ON orders.order_id = order_details.order_id  
 GROUP BY orders.order_date  
) AS daily_revenue;
```



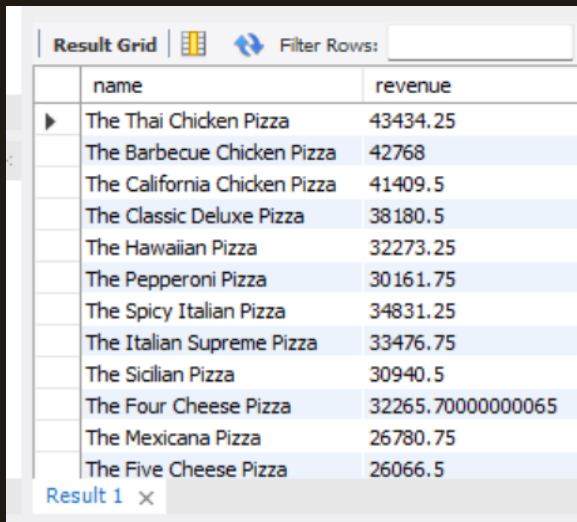
The screenshot shows a database query result grid with two columns: 'order_date' and 'cum_revenue'. The data is sorted by order_date from 2015-01-01 to 2015-01-12. The cumulative revenue starts at 2713.85 on 2015-01-01 and increases daily to 27781.7 on 2015-01-12. The interface includes a 'Result Grid' tab, a 'Filter Rows' button, and a 'Result 1' label at the bottom.

order_date	cum_revenue
2015-01-01	2713.8500000000004
2015-01-02	5445.75
2015-01-03	8108.15
2015-01-04	9863.6
2015-01-05	11929.55
2015-01-06	14358.5
2015-01-07	16560.7
2015-01-08	19399.05
2015-01-09	21526.4
2015-01-10	23990.350000000002
2015-01-11	25862.65
2015-01-12	27781.7

Which 3 pizzas lead revenue within each category?

q13.sql

```
SELECT name, revenue, category FROM
( SELECT category, name,
  revenue, RANK() OVER (PARTITION BY
category ORDER BY revenue DESC) AS rn
  FROM
( SELECT pizza_types.category, pizza_types.name,
      SUM(order_details.quantity * pizzas.price) AS revenue
  FROM pizza_types
  JOIN pizzas
    ON pizza_types.pizza_type_id = pizzas.pizza_type_id
  JOIN order_details
    ON order_details.pizza_id = pizzas.pizza_id
  GROUP BY pizza_types.category, pizza_types.name
) AS a
) AS b
WHERE rn <= 3;
```



The screenshot shows a 'Result Grid' window with a table of pizza names and their revenue. The table is sorted by revenue in descending order. The top 3 pizzas are highlighted in blue. The table has two columns: 'name' and 'revenue'.

name	revenue
The Thai Chicken Pizza	43434.25
The Barbecue Chicken Pizza	42768
The California Chicken Pizza	41409.5
The Classic Deluxe Pizza	38180.5
The Hawaiian Pizza	32273.25
The Pepperoni Pizza	30161.75
The Spicy Italian Pizza	34831.25
The Italian Supreme Pizza	33476.75
The Sicilian Pizza	30940.5
The Four Cheese Pizza	32265.70000000065
The Mexicana Pizza	26780.75
The Five Cheese Pizza	26066.5

Partitioning the ranking by category gives a top-3 per group — the analytical pattern behind every "leaderboard within a segment".

SQL Techniques Demonstrated

FOUNDATIONS

COUNT, SUM, AVG, ROUND
GROUP BY & ORDER BY
ORDER BY + LIMIT

RELATIONAL

Two & three-table JOINS
Quantity × price revenue
HOUR() time grouping

ADVANCED

Subqueries & derived tables
Percentage-of-total
Window funcs: SUM OVER,
RANK

Thirteen queries progressing from simple counts to partitioned window functions
— a complete arc from raw data to ranked business insight.



THANK YOU •

Let's talk data.

From raw orders to ranked insight —
this is how I turn questions into answers
with SQL.

- >full query set on GitHub
- >open to data analyst roles